

AEM Interview Preparation Guide

60+ Questions with Detailed Answers & Real-World Examples

Adobe Experience Manager | Cognizant Interview Prep

Section	Topics Covered	Questions
1	CMS & AEM Introduction	Q1–Q8
2	Maven & Project Setup	Q9–Q11
3	HTL / Sightly	Q12–Q14
4	Sling Models	Q15–Q18
5	Components & Templates	Q19–Q23
6	Touch UI Dialogs	Q24–Q30
7	OSGi	Q31–Q36
8	Sling Servlets	Q37–Q38
9	Client Libraries	Q39–Q40
10	Content & Experience Fragments	Q41–Q42
11	Advanced Topics	Q43–Q60

Section 1: CMS & AEM Introduction

Q1. What is AEM (Adobe Experience Manager)?

AEM is Adobe's enterprise Web Content Management platform built on Apache Sling, OSGi (Apache Felix), and JCR (Apache Jackrabbit Oak). It combines web content management, digital asset management, personalisation, and multi-channel delivery.

Example: Large enterprises like Nike, Vodafone, and Marriott use AEM to manage their digital experiences across web, mobile, and other channels.

Q2. What is the AEM Author-Publish-Dispatcher architecture?

Author: Where content creators log in to write and manage content. Changes are saved in JCR and replicated to Publish when ready.

Publish: Serves the live website to end users. Receives replicated content from Author.

Dispatcher: Apache module acting as reverse proxy and cache in front of Publish. Caches rendered HTML and adds a security layer.

Flow: Author → Replication → Publish → Dispatcher Cache → Browser

Example: A journalist writes an article on Author and clicks Activate. AEM replicates to all Publish nodes. Dispatcher cache flushes that URL; next visitor gets fresh content from cache.

Q3. What is JCR (Java Content Repository)?

JCR is the storage layer of AEM. All content — pages, assets, component data, configurations — is stored as nodes and properties in a hierarchical tree. AEM uses Apache Jackrabbit Oak as its JCR implementation.

Example: A page's content is stored at /content/mysite/en/home/jcr:content. The page title is a property jcr:title on that node.

Q4. What is Apache Sling and its role in AEM?

Sling is the web framework layer of AEM. It maps HTTP requests to JCR resources and finds the right script to render them. Sling follows REST principles — every URL maps to a content resource. Sling handles resource resolution, script resolution, request processing, and response rendering.

Q5. What is Sling Resource Resolution?

Sling decomposes a URL into: resource path, selectors, extension, suffix.

Example URL: /content/mysite/home.hero.html/extra

Path: /content/mysite/home | Selector: hero | Extension: html | Suffix: /extra

Sling finds the resource, then finds the rendering script by: /apps/[resourceType]/[selector].[ext].html → /apps/[resourceType]/[ext].html → fallback via sling:resourceSuperType.

Q6. What is CRXDE Lite?

CRXDE Lite is a browser-based IDE inside AEM (/crx/de/index.jsp) with direct access to the JCR repository. Used for debugging, exploring the repository, creating test nodes, and running JCR queries.

Key paths:

/apps → custom code | /content → authored content | /conf → editable template configs | /libs → AEM default (never edit directly)

Example: If a component isn't rendering an image, open CRXDE at /content/mysite/en/home/jcr:content/root/image and verify the fileReference property was saved correctly.

Q7. What is the OSGi Console?

The OSGi Web Console at /system/console is the admin UI for the OSGi container.

Bundles tab: manage JAR bundles, see state (Active/Installed/Resolved)

Components tab: view DS components and reference status

Configurations tab: edit OSGi configs at runtime

Services tab: view all registered OSGi services

Example: If your custom OSGi service isn't working, check the Components tab. If it shows 'unsatisfied reference', a required dependency is missing.

Q8. What is WYSIWYG in AEM?

WYSIWYG (What You See Is What You Get) means authors see content exactly as it appears on the live site while editing. AEM's Touch UI Sites Editor provides this — authors drag-and-drop components onto a live page preview with editable overlays.

Example: Clicking an Image component opens an inline toolbar to swap the image, add alt text, and crop — all while seeing the result in context on the page.

Section 2: Maven & Project Setup

Q9. What is Maven and how is it used in AEM?

Maven is a Java build and dependency management tool.

- Manages dependencies (AEM API jars, Sling, OSGi, third-party libs)
- Builds multi-module projects (core, ui.apps, ui.content, ui.frontend)
- Packages Java as OSGi bundles (.jar) and content as CRX packages (.zip)
- Deploys to AEM via HTTP

```
mvn clean install -PautoInstallSinglePackage
```

Q10. What is the AEM Maven Archetype and what modules does it generate?

Adobe's project scaffold template that generates a best-practice AEM project:

core — Java code: Sling Models, OSGi services, servlets

ui.apps — /apps content: components, templates, dialog XML, clientlibs

ui.content — /content: initial site content, editable template configs

ui.frontend — webpack/npm front-end build for CSS/JS

ui.config — OSGi configuration files per runmode

all — parent package wrapping all modules for single deployment

Q11. What is a POM file and what are the key sections?

POM (Project Object Model) is Maven's XML config file.

groupId/artifactId/version — project coordinates

packaging — jar, bundle, content-package, or pom

dependencies — required libraries with scope (provided, compile, test)

build > plugins — build tools like bnd-maven-plugin and filevault-package-maven-plugin

profiles — environment-specific config

IMPORTANT: AEM API jars must be 'provided' scope. If you use 'compile', they get bundled into your JAR causing conflicts at runtime.

Section 3: HTL / Sightly

Q12. What is HTL (Sightly) and how is it different from JSP?

HTL (HTML Template Language) is AEM's recommended server-side templating language replacing JSP.

Context-aware auto-escaping: automatically escapes output for HTML, URL, JS, CSS contexts — prevents XSS

No Java in templates: logic goes into Sling Models

Front-end friendly: plain HTML with data-sly-* attributes

Files use .html extension

Example: JSP: `<%=currentPage.getTitle()%>` — no escaping, XSS risk. HTL: `${properties.jcr:title}` — auto-escaped for HTML context, safe.

```
<div data-sly-use.model="com.mysite.models.HeroModel">
<h1>${model.title}</h1>

</div>
```

Q13. What are the key HTL block statements?

data-sly-use: instantiates a Sling Model/Use-class, binds to a variable

data-sly-test: conditional rendering — renders element only if truthy

data-sly-list: iterates over a collection; loop variable gives index, count, first, last

data-sly-include: includes another HTL script in same component context

data-sly-resource: includes a JCR resource rendered through its own component

data-sly-text: sets text content of an element

data-sly-attribute: dynamically sets or removes attributes

data-sly-unwrap: renders child content without the wrapper element

Example: data-sly-list for nav links, data-sly-test to show CTA only if author entered button text, data-sly-resource for rendering header/footer sub-resources.

Q14. What are HTL expression options (context)?

HTL's @ context option overrides automatic escaping:

context='html' — allows raw HTML output

context='uri' — escapes for URL attribute values

context='unsafe' — no escaping (use with extreme caution)

context='text' — HTML text escaping (default)

context='scriptString' — for inline JS strings

```

```

Section 4: Sling Models

Q15. What is a Sling Model?

A Sling Model is a POJO annotated with `@Model` that Sling auto-instantiates and injects values into, adapting from a Resource or `SlingHttpServletRequest`. It is the standard way to write business logic for AEM components.

Example: A 'Card' component dialog has fields: title, linkPath, imageReference. The Sling Model reads all via `@ValueMapValue` and exposes clean getters for HTL.

```
@Model(adaptables=Resource.class,
defaultInjectionStrategy=DefaultInjectionStrategy.OPTIONAL)
public class CardModel {
    @ValueMapValue private String title;
    @ValueMapValue private String linkPath;
    public String getTitle() { return title; }
    public String getLinkPath() { return linkPath; }
}
```

Q16. What are the key Sling Model annotations?

`@Model` — marks the class as a Sling Model; defines adaptables, adapters, defaultInjectionStrategy

`@ValueMapValue` — injects a JCR property from the content node

`@Inject` — general-purpose injection; Sling tries multiple injectors in order

`@OSGiService` — injects a registered OSGi service

`@Self` — injects the adaptable itself (Resource or Request)

`@SlingObject` — injects ResourceResolver, SlingHttpServletRequest/Response

`@ScriptVariable` — injects scripting context objects: currentPage, currentNode, properties

`@ChildResource` — injects a child node as a Resource

`@RequestAttribute` — injects a request attribute (Request adaptable only)

`@PostConstruct` — method runs after all injections are complete

Q17. What is @PostConstruct in Sling Models?

`@PostConstruct` marks a method that runs after all field injections are complete. Used to compute derived values, validate data, or initialise complex fields.

Example: After sku and productService are injected, @PostConstruct calls productService.getPrice(sku) and stores the result in a price field that HTL can then display.

```
@PostConstruct
protected void init() {
    this.price = productService.getPrice(sku);
}
```

Q18. Resource vs SlingHttpServletRequest adaptable — what is the difference?

Resource adaptable: Model gets only the content node's JCR properties. Use when you only need dialog data. Testable, cacheable.

SlingHttpServletRequest adaptable: Model has access to query parameters, headers, selectors, session, request attributes. Use when logic depends on the HTTP request.

Best practice: Default to Resource. Use Request adaptable only when you genuinely need request-specific data.

Example: SearchModel needs the 'q' query parameter from the URL → Request adaptable. TextModel only needs 'text' and 'color' from the dialog → Resource adaptable.

Section 5: Components & Templates

Q19. What is an AEM Component? Describe its folder structure.

An AEM Component is a modular, reusable content block that authors drag onto pages.

```
/apps/mysite/components/card/
■■■■ .content.xml (title, componentGroup, sling:resourceSuperType)
■■■■ card.html (HTL rendering script)
■■■■ _cq_dialog/
■ ■■■■ .content.xml (Touch UI authoring dialog)
■■■■ _cq_editConfig.xml (edit bar config)
■■■■ _cq_design_dialog/
■ ■■■■ .content.xml (template-level dialog)
■■■■ clientlibs/
■■■■ .content.xml (categories, dependencies)
■■■■ card.css
■■■■ card.js
```

Q20. What is sling:resourceSuperType (component inheritance)?

sling:resourceSuperType on a component node points to a parent component. AEM inherits HTL scripts, dialog fields, and edit config from the parent. If your component lacks a rendering script, AEM walks up the super type chain to find one.

Best practice: Extend Adobe Core Components via sling:resourceSuperType instead of building from scratch.

Example: sling:resourceSuperType = core/wcm/components/image/v3/image — your custom Hero inherits image resolution, DAM integration, and lazy loading for free.

```
<jcr:root
jcr:primaryType="cq:Component"
sling:resourceSuperType="core/wcm/components/image/v3/image"
jcr:title="Hero Banner"
componentGroup="MyProject - Content" />
```

Q21. What is Overlay vs Override in AEM?

Overlay: Copy a /libs node to the same path under /apps. Sling Resource Merger resolves /apps over /libs. Use to modify AEM built-in behaviour without touching /libs.

Override (inheritance): Use sling:resourceSuperType to extend a component, overriding specific scripts while inheriting the rest.

Rule: Never modify /libs directly — always overlay to /apps.

Example: The default Page Properties dialog lacks a 'Campaign Code' field. Overlay /libs/foundation/components/page/cq:dialog to /apps/foundation/components/page/cq:dialog and add the field.

Q22. What is Static Templates vs Editable Templates?

Static Templates: Defined under /apps. Fixed structure — every structural change needs developer deployment. Authors cannot configure them.

Editable Templates (recommended): Managed under /conf/[project]/settings/wcm/templates via the Template Editor UI.

Three modes:

Structure — locked layout (header/footer) authors cannot change

Initial Content — default content authors CAN modify

Layout — responsive breakpoint settings

Policies control which components are allowed in containers.

Example: A 'Landing Page' editable template locks the header in Structure mode. Marketing can't delete it. The content zone is open so authors can add hero banners, text blocks, and CTAs freely.

Q23. What is the AEM Style System?

The Style System lets content authors apply CSS class variations to components without developer involvement.

1. Developer registers CSS classes as named styles in the component policy
2. Template author enables styles in the template policy
3. Content author clicks the paintbrush icon and picks a style
4. AEM adds the CSS class to the component wrapper div

Example: A Teaser component has styles: Default, Dark Background, Horizontal Layout. Author selects 'Dark Background' — AEM adds 'cmp-teaser--dark' to the wrapper. No code deployment needed.

Section 6: Touch UI Dialogs

Q24. What is a Touch UI Dialog? Explain Granite UI.

Granite UI is the server-side rendering framework powering AEM's Touch UI. It provides form field components (textfield, pathbrowser, checkbox, select, multifield, etc.).

A Touch UI Dialog is a JCR node structure under `_cq_dialog/`. Fields use `sling:resourceType` pointing to Granite UI components. The 'name' property maps the field to the JCR property (must start with `./`).

Example: A 'Card' dialog has two tabs: 'Content' (title, description, linkPath, imageReference) and 'Settings' (colorTheme select, hideOnMobile checkbox).

```
<title
sling:resourceType="granite/ui/components/coral/foundation/form/textfield"
fieldLabel="Card Title"
name="./title" />
```

Q25. What Granite UI field types are commonly used?

- form/textfield — single-line text
- form/textarea — multi-line text
- form/pathbrowser — path picker for pages/assets
- form/select — dropdown
- form/checkbox — boolean checkbox
- form/numberfield — numeric input
- form/datepicker — date picker
- form/colorfield — color picker
- form/fileupload — file upload
- form/multifield — repeating group of fields
- form/hidden — hidden field with fixed value

Q26. What is a Multifield in AEM dialogs?

Multifield allows authors to add multiple entries of a repeated structure.

Simple Multifield: Multiple values of one field (stored as multi-value JCR property).

Composite Multifield: Each entry is a group of fields (e.g., label + URL + icon). Set `composite={Boolean}true`. Each entry stored as a child node. In Sling Model, iterate child nodes as List.

Example: A 'Navigation Links' composite multifield has per-entry: Label, URL, Open-in-new-tab checkbox. An author adds 5 navigation entries through the dialog.

Q27. What are Listeners in AEM dialogs?

Listeners are client-side JavaScript event handlers attached to dialog fields via a `cq:listeners` child node. They react to events like `afterrender`, `change`, `click`, `loadcontent` to create dynamic dialog behaviour — show/hide fields, populate dropdowns, validate related fields.

Example: A 'Banner Type' dropdown has a change listener — when the author selects 'video', the image picker hides and a video URL field shows.

Q28. What are Validators in AEM dialogs?

Validators enforce field value rules before the dialog can be saved.

Built-in: required attribute, minlength, maxlength, regex via validation property.

Custom Foundation validators: Register via JavaScript using `foundation.validation.validator`. Inline error message appears below the field and blocks Save.

Example: A 'Promo Code' field must match `[A-Z]{3}-[0-9]{4}`. A custom validator checks the pattern and shows: 'Code must be in format ABC-1234' if invalid.

Q29. What is `cq:EditConfig`?

`_cq_editConfig.xml` configures the edit behaviour of a component in the AEM page editor:

`cq:actions` — which actions appear in the component toolbar (edit, delete, copymove)

`cq:emptyText` — placeholder shown when component has no content

`cq:inplaceEditing` — enables inline text editing without opening a dialog

`cq:listeners` — server-side and client-side edit event handlers (afterEdit, afterInsert, afterDelete)

`cq:dropTargets` — defines drag-and-drop zones for assets

Q30. What is a Design Dialog in AEM?

`_cq_design_dialog` is configured at the template/design level, not per page instance. Settings here apply globally to all instances of the component on pages using that template. Accessed via Design Mode or via the template policy.

Example: A 'Text' component's design dialog lets the template author configure which paragraph formats (H2, H3, blockquote) are available in the RTE — applying to every Text component on all pages using that template.

Section 7: OSGi

Q31. What is OSGi? Explain bundles, components, and services.

OSGi is a dynamic module system for Java. AEM runs on Apache Felix (an OSGi framework).

Bundle: A JAR with OSGi metadata. Can be started, stopped, updated at runtime without restarting AEM.

Component: Java class annotated `@Component` managed by the DS (Declarative Services) container.

Service: Interface registered in the OSGi service registry, consumed by other components by interface type — decoupling consumer from implementation.

Example: You create an `EmailService` interface. `SendGridEmailServiceImpl` implements it. Your Sling Model injects it via `@OSGiService` — it has no knowledge of `SendGrid`. Swap to `Mailchimp` by deploying a new implementation, no consumer changes needed.

Q32. What is the OSGi component lifecycle?

1. Unsatisfied: Required references unavailable. Component not activated.
2. Activate (`@Activate`): All dependencies met. Initialise resources, read config, start threads, open connections.
3. Active: Running, services available to consumers.
4. Modified (`@Modified`): Config changed while active — called instead of full restart if annotated.
5. Deactivate (`@Deactivate`): Bundle stopping or dependency gone. Clean up resources here.

Example: A `CacheService` reads TTL from OSGi config in `@Activate` and builds a Guava Cache. In `@Deactivate` it calls `cache.invalidateAll()`. Admin changes TTL in Felix Console — `@Modified` rebuilds the cache. No AEM restart required.

```
@Activate
protected void activate(CacheConfig cfg) {
    cache = CacheBuilder.newBuilder()
        .expireAfterWrite(cfg.ttlMinutes(), TimeUnit.MINUTES)
        .build();
}
@Deactivate
protected void deactivate() { cache.invalidateAll(); }
```

Q33. How do you configure an OSGi service?

File-based (recommended): `.cfg.json` files in `/apps/[project]/osgiconfig/config.[runmode]/`. Checked into source control. Deployed with content package.

Node-based: `slings:OsgiConfig` nodes in CRXDE. Less common.

OSGi Console: Runtime-only changes. May not persist after restart.

Runmode resolution: `config.[runmode]` wins over plain config. Use for environment-specific configs: `config.author`, `config.publish`, `config.dev`.

Example: You need different SMTP settings per environment. Place `config.author/EmailService.cfg.json` and `config.publish/EmailService.cfg.json`. AEM picks the right one based on its runmode.

```
// com.mysite.EmailServiceImpl.cfg.json
{
  "smtp.host": "smtp.sendgrid.net",
  "smtp.port": 587,
```

```
"smtp.from": "noreply@mysite.com"
}
```

Q34. What is a ServiceFactory in OSGi?

A ServiceFactory creates a different service instance per consuming bundle, unlike a regular service that provides one shared instance. Used when the service needs per-consumer state or configuration.

Annotate with `@Component(servicefactory=true)`. The `getService(Bundle)` method creates a new instance per bundle, and `ungetService()` cleans it up when the bundle releases the service.

Q35. What is Declarative Services (DS) in OSGi?

Declarative Services is the OSGi specification for defining, registering, and consuming services via XML descriptors or annotations. AEM uses DS annotations from the `org.osgi.service.component.annotations` package.

Key annotations: `@Component`, `@Service` (deprecated, replaced by service param in `@Component`), `@Reference` (for injecting OSGi services inside non-Sling-Model classes), `@Activate`, `@Deactivate`, `@Modified`.

Q36. What is the difference between a JAR and a Bundle in OSGi?

A JAR is a standard Java archive containing compiled classes and resources.

A Bundle is a JAR with additional OSGi metadata in its MANIFEST.MF file:

Bundle-SymbolicName — unique bundle identifier

Bundle-Version — version string

Export-Package — packages made available to other bundles

Import-Package — packages this bundle requires from others

OSGi uses this metadata to manage dependencies and class loading between bundles at runtime.

Section 8: Sling Servlets

Q37. What is a Sling Servlet and how do you register one?

A Sling Servlet is an OSGi component handling HTTP requests for specific resources or paths, integrated into the Sling pipeline.

By resource type (recommended): Handles requests for any resource with a specific sling:resourceType.

By path (use sparingly): Handles requests to a specific URL path — bypasses Sling resource resolution.

Extend `SlingSafeMethodsServlet` (GET only) or `SlingAllMethodsServlet` (GET + POST + DELETE + PUT).

Example: A Search API needs JSON at `/content/mysite/en.search.json?q=shoes`. Register by resourceType, extension 'json', selector 'search'. AEM resolves the page resource and routes to the servlet.

```
@Component(service=Servlet.class)
@SlingServletResourceTypes(
    resourceTypes="mysite/components/search",
    methods="GET",
    extensions="json",
    selectors="search"
)
public class SearchServlet extends SlingSafeMethodsServlet {
    @Override
    protected void doGet(SlingHttpServletRequest req,
        SlingHttpServletResponse resp) throws IOException {
        String q = req.getParameter("q");
        resp.setContentType("application/json");
        resp.getWriter().write("{\"query\":\"" + q + "\"}");
    }
}
```

Q38. What is the Sling Servlet resolver and default servlet?

When Sling resolves a request, it looks for a matching servlet in order of specificity:

1. Servlet for exact resourceType + selector + extension
2. Servlet for resourceType + extension (no selector)
3. Default servlet for the extension (`GET.html.servlet`)
4. AEM's default `DefaultGetServlet`

The default GET servlet renders the resource's properties as JSON. This is why `/content/mysite/home.json` works out of the box.

Section 9: Client Libraries

Q39. What are Client Libraries (ClientLibs) in AEM?

ClientLibs are AEM's system for managing, combining, and serving CSS and JS.

category: String identifier to reference the library

dependencies: Other categories that must load first

embed: Pull another library's files into this one (bundling)

allowProxy: Must be true for Publish security (serves via /etc.clientlibs)

Example: Site has `mysite.base` (global CSS), `mysite.components.hero` (hero CSS/JS), `mysite.components.nav` (nav JS). Page `clientlib` embeds `mysite.base`. Each component `clientlib` declares `mysite.base` as a dependency so it always loads first.

```
<sly data-sly-use.clientlib="/libs/granite/sightly/templates/clientlib.html">
<sly data-sly-call="${clientlib.css @ categories='mysite.base'}" />
<sly data-sly-call="${clientlib.js @ categories='mysite.base'}" />
</sly>
```

Q40. What is minification and debugging of ClientLibs?

Minification: AEM automatically minifies CSS and JS in production mode (`debug=false`). Controlled via OSGi config for `HtmlLibraryManager`.

Debugging: Append `?debugClientLibs=true` to any page URL to load un-minified, individual files.

GZip compression: Enabled via `HtmlLibraryManager` config. AEM serves gzipped clientlibs for supporting browsers.

Section 10: Content & Experience Fragments

Q41. What are Content Fragments in AEM?

Content Fragments (CF) are structured, presentation-free content stored in /content/dam. Defined by Content Fragment Models (schema). Used for headless/omnichannel delivery via AEM's Content Services REST API or GraphQL API. The same CF can be consumed by a website, mobile app, digital signage, or voice assistant.

Example: A 'Product' CF model has: productName, sku, shortDescription, specifications (multifield). The website, iOS app, and in-store kiosk all fetch it via API and render it in their own way.

Q42. What are Experience Fragments in AEM and how are they different from Content Fragments?

Experience Fragments (XF) are reusable authored fragments bundling content AND layout/presentation. Made of AEM components in a page-like structure. Rendered as HTML. Can be exported to Adobe Target for A/B testing. Stored under /content/experience-fragments.

Key difference: CF = content only, channel-agnostic. XF = content + presentation, channel-specific.

Example: An 'Anniversary Sale Banner' XF has a background image, headline, countdown timer, and CTA button. Reused across 10 campaign pages and exported to Adobe Target for A/B testing.

Section 11: Advanced Topics

Q43. What is an RTE (Rich Text Editor) and how do you configure it?

The RTE is AEM's WYSIWYG text editor inside component dialogs, allowing authors to format text.

Configuration:

Toolbar plugins configured via template policy or design dialog

Allowed HTML tags whitelist

Custom styles dropdown (applies CSS classes to selected text)

Paste mode (plain text or formatted)

Common plugins: bold, italic, underline, lists, link, table, paraformat, spellcheck, undo

Example: A 'Blog Post Body' RTE policy enables: bold, italic, lists, links, H2, H3. Disables tables and underline to maintain design consistency.

Q44. What is the difference between cq:Page and cq:PageContent?

cq:Page: Primary node type for AEM pages stored under /content. Represents the page itself. Its child jcr:content node holds all page data.

cq:PageContent: Node type of the jcr:content child node. Holds page properties (jcr:title, sling:resourceType, cq:template) and the component tree (layout containers, component data).

Q45. What is Replication in AEM?

Replication transfers content from Author to Publish instances. When an author clicks 'Activate', AEM's Replication Agent sends the content package to configured Publish endpoints.

Types:

Standard Replication: Author to Publish (forward replication)

Reverse Replication: Publish to Author (for user-generated content)

Flush Agent: Dispatcher cache invalidation — sends HTTP DELETE requests to Dispatcher to flush cached files when content is published

Q46. What is a Workflow in AEM?

AEM Workflows automate multi-step content processes. A workflow consists of steps (participants, processes, splits, joins) that route content through approval or processing stages.

Common workflows:

Request for Activation: Content goes through approval before publishing

DAM Update Asset: Runs on asset upload — generates renditions, extracts metadata, creates thumbnails

Page Version Purge: Removes old page versions to manage repository size

Example: A 'Content Approval' custom workflow: Author submits page → Editor reviews → Legal reviews → SEO reviews → Published. Each step has a participant step assigning a task to the right user group.

Q47. What are Workflow Launchers?

Workflow Launchers automatically trigger a workflow when a specific JCR event occurs at a given path. Configured under Tools → Workflow → Launchers.

Events: Node Created, Node Modified, Node Removed, Property Changed

Example: A launcher on /content/mysite for the 'Node Modified' event triggers the 'Request for Activation' workflow every time a content author modifies a page — automatically sending it for approval without manual action.

Q48. What is Dispatcher caching and how is cache invalidation handled?

Dispatcher caches rendered HTML responses on the web server's file system. On subsequent requests for the same URL, the cached file is served without hitting Publish.

Cache invalidation: When an author publishes a page, AEM's Flush Agent sends an HTTP request with CQ-Action: Delete and the page path to the Dispatcher. Dispatcher deletes the cached file. The next request re-renders from Publish and re-caches.

Dispatcher rules: /filter defines which URLs are allowed or blocked. /cache defines which paths to cache and TTL.

Q49. What is Sling URL Mapping and how do Vanity URLs work?

Sling URL Mapping maps friendly external URLs to internal JCR paths. Configured in /etc/map or via OSGi Apache Sling URL Map configuration.

Vanity URLs: A shorter URL set directly on the page (jcr:content/sling:vanityPath property).

Example: Example: /campaign/summer-sale mapping to /content/mysite/en/campaigns/2025/summer-sale.html. Configured in Page Properties under the 'Vanity URL' field.

Q50. What is the difference between nt:unstructured and cq:Page?

nt:unstructured: Flexible JCR node type accepting any property or child node. Used for component data (dialog field values), OSGi configs, and /apps configuration nodes. No schema enforcement.

cq:Page: Structured node type for AEM pages under /content. Enforces a jcr:content child node. AEM's Sites console recognises these as pages — they appear in the site tree, support activation/deactivation, page properties, etc.

Q51. What is the Externalizer service in AEM?

The Externalizer OSGi service converts relative JCR paths to absolute external URLs. Configured with named domains (local, author, publish) in the Day CQ Link Externalizer OSGi config.

Example: Used in email templates, sitemaps, and any code that generates absolute URLs for external consumption.

```
@OSGiService
private Externalizer externalizer;

String url = externalizer.publishLink(resourceResolver, "/content/mysite/en/home");
// Returns: https://www.mysite.com/content/mysite/en/home.html
```

Q52. What is the QueryBuilder API in AEM?

QueryBuilder is a JCR query API specific to AEM providing a simplified predicate-based syntax to search the JCR repository. Easier than writing raw SQL2 or XPATH queries.

Example: Accessible at /libs/cq/search/content/querydebug.html in the Query Debugger UI for testing.

```
Map map = new HashMap<>();
map.put("type", "cq:Page");
```

```
map.put("path", "/content/mysite");
map.put("property", "jcr:content/sling:resourceType");
map.put("property.value", "mysite/components/landingpage");
map.put("p.limit", "10");
Query query = queryBuilder.createQuery(PredicateGroup.create(map), session);
```

Q53. What is a RunMode in AEM?

RunModes are tags applied to an AEM instance that determine its role and environment. They control which OSGi configurations and content packages are applied.

Default runmodes: author, publish

Environment runmodes: dev, stage, prod

Custom runmodes: any string you define

Example: An instance started with -Dsling.run.modes=publish,prod loads configs from config.publish, config.prod, and config.publish.prod — the most specific combination wins.

Q54. What is the AEM Package Manager?

Package Manager (/crx/packmgr) is AEM's UI and API for building, uploading, installing, and distributing CRX content packages (.zip files containing JCR content).

Packages include:

Filters: define which JCR paths are included

Content: the actual nodes/properties exported as XML

Used to: deploy code, migrate content, back up and restore repository state, share component libraries.

Q55. What are Core Components in AEM?

Adobe Core Components are production-ready, best-practice AEM components available open source. They implement common patterns (Text, Image, Title, Teaser, Navigation, Breadcrumb, Search, Carousel, etc.) with:

Full HTL implementation

Sling Model-backed business logic

Editable template and Style System support

Accessibility compliance (WCAG 2.1)

BEM CSS conventions

Custom components should extend Core Components via sling:resourceSuperType.

Q56. What is the Sling Model Exporter?

Sling Model Exporter serializes a Sling Model to JSON (or XML) automatically, enabling a component to expose its data as a REST API endpoint. Used in AEM as a Headless CMS and for SPA architecture.

Example: Requesting /content/mysite/home/jcr:content/root/card.model.json returns the model as JSON.

```
@Model(adaptables=Resource.class, resourceType="mysite/components/card")
@Exporter(name="jackson", extensions="json")
public class CardModel {
    @ValueMapValue private String title;
    public String getTitle() { return title; }
}
```

Q57. What is AEM's Digital Asset Management (DAM)?

AEM DAM (stored at /content/dam) is the built-in digital asset management system. It stores images, videos, PDFs, and other files.

Key features:

Asset upload with automatic rendition generation via DAM Update Asset workflow

Metadata schemas and tagging (Smart Tags via Adobe Sensei AI)

Asset search with faceted filtering

Asset sharing and download links

Integration with Creative Cloud (Adobe Asset Link)

DRM and rights management

Q58. What is a Rendition in AEM DAM?

A Rendition is a representation of an asset in a specific format or size. When an asset is uploaded, the 'DAM Update Asset' workflow generates renditions automatically:

original — the uploaded file

cq5dam.web.1280.1280.jpeg — web-optimised version

cq5dam.thumbnail.48.48.png — small thumbnail

cq5dam.thumbnail.319.319.png — medium thumbnail

Custom renditions can be added to the workflow profile for project-specific needs.

Q59. What is i18n (internationalisation) support in AEM?

AEM provides i18n support for multi-language sites via the I18n API and translation dictionaries.

Translation dictionaries: JCR nodes under /apps/[project]/i18n/[language].json mapping keys to translated strings.

In HTL: \${properties.title @ i18n} auto-translates using the current page language.

In Java: I18n i18n = new I18n(request); String label = i18n.get("search.button");

AEM also supports Translation Integration Frameworks connecting to Microsoft Translator, DeepL, etc.

Q60. What is the difference between /libs and /apps?

/libs: AEM's default code — Core Components, Foundation Components, Granite UI, platform servlets, workflows. Managed by Adobe. Updated on AEM upgrades. Never modify directly.

/apps: Your custom code — project components, templates, configurations, overlays. Under your team's control. Sling Resource Merger resolves /apps over /libs for any overlapping paths, so /apps always wins.

This separation ensures your customisations survive AEM version upgrades.

Quick Reference: Most Asked in Cognizant AEM Interviews

Topic	Questions to Focus On
Author-Publish-Dispatcher	Q2 — draw the architecture, explain each layer
Sling Resource Resolution	Q5 — how a URL maps to a script
Sling Model annotations	Q15, Q16, Q17 — write code from memory
Resource vs Request adaptable	Q18 — when to use each
HTL block statements	Q13 — know all data-sly-* statements
OSGi lifecycle	Q32 — explain @Activate, @Deactivate, @Modified
OSGi configuration	Q33 — file-based vs node-based, runmode resolution
Overlay vs Override	Q21 — when to use each, never modify /libs
Content vs Experience Fragments	Q41, Q42 — the classic difference question
Sling Servlet registration	Q37 — write the annotations from memory
Touch UI Dialog structure	Q24 — explain Granite UI field types and name property